HIREFIT AI • COMPREHENSIVE PROJECT HANDBOOK

HireFit AI: Complete Architecture, Codebase & Presentation Handbook

Adaptive Target-Company & Role-Specific Interview Simulator
Detailed Technical Breakdown, Package Specifications, Code Walkthrough & Master Viva Defense Q&A

Project Name	HireFit AI (Adaptive Interview Simulator)
Architecture	FastAPI (Async Python 3.10+) + MongoDB + React 19 (TypeScript + shadcn/ui)
Key Innovations	Company Intel Scraping, Sigmoid Selection Probability, 5 Role Tracks, ATS Optimizer
Evaluation Focus	Capstone Project Review, Viva Voce & Technical Defense

1. Executive Summary & Problem Statement

HireFit AI is an advanced, text-based adaptive interview simulation platform engineered to address the growing asymmetry in contemporary technology and corporate recruitment. Traditional mock interview platforms rely on static LeetCode coding problems or generic behavioural templates that do not reflect the actual company culture, distributed system architectures, or quantitative expectations of specific employers such as Stripe, Google, Amazon, or Salesforce.

The 3 Core Flaws in Modern Interview Preparation

- **Lack of Company Context:** Candidates prepare generic answers without knowing the target company's actual tech stack, operational constraints (e.g. Stripe's idempotency requirements or Google's tail latency tolerances), or cultural rubrics.
- **Uncalibrated Assessment & Hallucinatory Feedback:** Most AI mock tools output arbitrary ratings without mathematical validation or benchmark standards, leaving candidates uncertain about real-world hiring odds.
- **Disconnect from ATS & Resumes:** Existing simulators evaluate answers in isolation from the candidate's actual resume, failing to spot skill gaps or optimize bullet points for Applicant Tracking Systems (ATS).

The HireFit AI Solution

HireFit AI ingests three core inputs: (1) Candidate Resume (PDF/DOCX), (2) Target Job Role & Description, and (3) Target Employer Name. It then triggers a live asynchronous web scraping engine to extract the company's real-time tech stack, culture, and valuation, maps candidate readiness against company competencies, conducts a 20-question, 2-round structured assessment (Aptitude & Sizing + Domain Scenarios), and generates a statistically calibrated Offer Selection Probability alongside an actionable ATS Resume Optimizer.

2. Comprehensive Package & Dependencies Directory

2.1 Backend Dependencies (Python 3.10+)

The backend is built as a modular FastAPI asynchronous service. Below is every package in `requirements.txt` and its exact technical purpose:

Backend Package	Technical Purpose & Role in Architecture
fastapi (>=0.110.0)	High-performance asynchronous web API framework. Handles routing, dependency injection, automatic OpenAPI/Swagger documentation generation at `/docs`, and native async/await request concurrency.
uvicorn (>=0.28.0)	Lightning-fast ASGI (Asynchronous Server Gateway Interface) web server implementation for Python. Runs the FastAPI application with auto-reload capabilities.
pydantic (>=2.6.0)	High-speed data validation and parsing using Python type hints. Enforces strict schema validation on all API payloads (`SessionCreateResponse`, `QuestionItem`, `EvaluationReportResult`) with zero hallucination.
pydantic-settings (>=2.2.0)	Configuration management for Pydantic v2. Automatically loads environment variables from `.env` into typed Settings objects (`MONGODB_URL`, `LLM_API_KEY`, scoring weights).
motor (>=3.3.0)	The official asynchronous, non-blocking Python driver for MongoDB. Allows FastAPI async handlers to execute non-blocking database queries without stalling the event loop.
pymongo (>=4.6.0)	Synchronous MongoDB client dependency required by Motor for low-level BSON serialization and connection pooling.
httpx (>=0.27.0)	Next-generation HTTP client for Python with native async support and HTTP/2 capabilities. Used in the web scraping pipeline to fetch company public pages with strict timeout enforcements.
beautifulsoup4 (>=4.12.0)	HTML parsing library. Extracts text content, clears noisy script/style tags, and isolates corporate descriptions, hero tags, and tech stack references from scraped HTML.
pdfplumber (>=0.10.0)	Deep layout-aware PDF extraction library. Extracts text lines, tables, and sections from candidate resumes without character truncation.
python-docx (>=1.1.0)	Microsoft Word (.docx) document ingestion and generation library. Used for parsing uploaded DOCX resumes and exporting technical reports.
python-multipart (>=0.0.9)	Streaming multipart form-data parser required by FastAPI to receive uploaded resume files and text form fields in a single HTTP POST request.
pytest & pytest-asyncio	Automated unit and integration testing framework. Enables non-blocking asynchronous testing of FastAPI routers, scoring rubrics, and scraper functions.

2.2 Frontend Dependencies (React 19 + TypeScript + Vite)

The frontend is built with React 19, Vite, and Tailwind CSS v4, styled exclusively using standard shadcn/ui components:

Frontend Library	Technical Purpose & Role in Architecture
react & react-dom (19.2.x)	The core UI library. Leverages React 19 concurrent features, improved hydration, and optimized DOM rendering.
vite (8.3.x)	Next-generation frontend tooling and build server providing instantaneous Hot Module Replacement (HMR) and optimized Rollup production bundling.
typescript (~6.0.x)	Static typing language providing compile-time type safety across API interfaces, component props, and complex state models.
@tailwindcss/vite & tailwindcss (v4.3.x)	Modern utility-first CSS framework. Version 4 includes native Vite integration, lightning CSS processing, and streamlined design tokens.
@radix-ui/react primitives	Accessible, unstyled UI primitives powering shadcn/ui: Dialog, Tabs, Progress, ScrollArea, Separator, Slot, and Label. Ensures full WCAG 2.1 AA keyboard navigation.
lucide-react (1.48.x)	High-performance SVG icon collection used consistently throughout navigation, metric cards, badges, and alerts.
sonner (2.0.x)	Smooth, toast notification library. Provides instant feedback for actions like copying AI bullet points, timer warnings, and scraping status.
clsx & tailwind-merge	Utility functions for conditionally joining class names and intelligently merging conflicting Tailwind classes without CSS specificity bugs.
react-router-dom (7.18.x)	Client-side routing engine managing navigation between Home, CompanyDetail, Profile, Setup, Interview, and Result pages.

3. Comprehensive Codebase Architecture & File Walkthrough

3.1 Backend Modules (backend/app/)

> **1. backend/app/main.py**
>
> Central entry point of the FastAPI backend. Implements an asynchronous lifespan context manager (`lifespan`) that connects to MongoDB on startup via `connect_to_mongo()` and closes the connection pool gracefully on shutdown. Configures CORS middleware, mounts all sub-routers under `/api` , and exposes `/api/health` for monitoring.

> **2. backend/app/core/config.py**
>
> Central configuration class using `pydantic_settings.BaseSettings` . Defines environment variables with sensible defaults: `MONGODB_URL` , `LLM_PROVIDER` , `LLM_MODEL` , CORS origins, file size limits (5MB), and the 5 mathematical scoring weights (Resume Fit: 0.25, Technical: 0.35, Communication: 0.15, Problem Solving: 0.15, Company Awareness: 0.10).

3. backend/app/core/db.py

Manages asynchronous MongoDB connectivity using Motor. Initializes the database instance and provides a `get_database()` accessor. If MongoDB is not running locally, the application catches the connection timeout and seamlessly falls back to an in-memory dictionary cache (`in_memory_sessions`), ensuring zero crashes.

4. backend/app/core/security.py

Enforces security guardrails: (1) `validate_resume_file()` verifies file extension (.pdf, .docx, .txt) and 5MB limit. (2) `sanitize_user_answer()` strips prompt injection exploits like 'ignore previous instructions' and escapes HTML/script tags. (3) PII scrubbing cleans confidential candidate identification before LLM calls.

5. backend/app/core/llm.py

Provider-agnostic LLM interface supporting OpenRouter, Anthropic, Google Gemini, and OpenAI. Crucially features a robust `MockLLMClient` fallback: if no API key is configured or network is offline, the mock client generates structured, deterministic JSON evaluations conforming to Pydantic schemas.

6. backend/app/services/company_scraper.py

Real-time company intelligence ingestion pipeline. Features `is_allowed_by_robots()` using `urllib.robotparser` to honor robots.txt rules for 'HireFitBot'. Uses `httpx.AsyncClient` with a 4.0-second timeout to fetch pages, cleans DOM nodes via BeautifulSoup4, and estimates company worth, tech stack, and culture.

7. backend/app/services/question_bank.py

Contains role-detection heuristics (`detect_role_category`) and pre-calibrated question suites for 5 specialized industry tracks: Tech (Distributed Systems), Sales (MEDDPICC), PM (Metrics/TAM), Data Science (MLOps/Skew), and Finance (DCF/LBO). Each track provides 20 structured questions divided into Round 1 (Aptitude) and Round 2 (Applied Scenarios).

8. backend/app/services/interview_engine.py

Core interview execution logic. Generates full 20-question assessments upfront (`generate_all_questions_for_session`). Provides `evaluate_batch_answers()` which evaluates all candidate submissions concurrently using Python's `asyncio.gather(*eval_tasks)` for sub-second evaluation.

9. backend/app/services/scorer.py

Implements the mathematical evaluation rubrics. Calculates candidate-role skill overlap, computes weighted category scores across turns, calculates the logistic sigmoid offer selection probability, and generates actionable ATS resume optimization bullet points.

10. backend/app/routers/ (session.py, interview.py, company.py, score.py)

- `POST /api/sessions` : Creates session, parses resume bytes, triggers background context analysis.
- `GET /api/sessions/{id}/questions` : Returns all 20 calibrated interview questions upfront for smooth client-side navigation.
- `POST /api/sessions/{id}/submit-all` : Submits all answers and evaluates them concurrently via asyncio.gather.
- `GET /api/sessions/{id}/result` : Compiles and returns final multi-dimensional scorecard and ATS report.
- `POST /api/companies/scrape` : Scrapes any user-entered company website and returns a full hiring rubric with financial valuation.

3.2 Frontend Modules (frontend/src/)

1. frontend/src/pages/Home.tsx

Primary employer exploration hub. Displays preset company cards (Google, Stripe, Amazon, Salesforce, Goldman Sachs) plus custom scraped companies. Includes track filter tabs, real-time search, and '+ Scrape Company' modal.

2. frontend/src/pages/CompanyDetail.tsx

Employer dossier and readiness screen. Displays company financial valuation, culture, and 2-round hiring stages. Computes real-time 'Resume Readiness' score by cross-referencing candidate skills against company competencies.

3. frontend/src/pages/Interview.tsx

The core testing environment. Key features: (1) 30-minute countdown timer with pause/resume and color-coded alerts. (2) Interactive Learning Mode toggle: Practice Mode (instant concept rationale) vs Exam Mode (strict test conditions). (3) Bookmark / Flag for Review button. (4) 1..20 jumpable question grid. (5) LocalStorage state caching.

4. frontend/src/pages/Result.tsx

Comprehensive scorecard. Displays Overall Score (/100), Calibrated Offer Probability (%), and ATS Resume Match (%). Provides 3 tabs: Performance Overview, Question Review (all 20 questions with verified answer keys), and ATS Resume Optimizer (with 1-click copyable bullets). Includes one-click 'Download / Print PDF' button.

4. Mathematical Formulas & Scoring Algorithms

4.1 Multi-Dimensional Weighted Performance Score

The candidate's overall score `S_overall` is a normalized linear combination of 5 evaluation categories:

```
S_overall = 0.25 * S_resume_fit + 0.35 * S_technical + 0.15 * S_communication + 0.15 *
S_problem_solving + 0.10 * S_company
```

Where each category score is bounded in [0, 100]. This rubric ensures technical accuracy is prioritized (35%) while guaranteeing architectural trade-offs, communication clarity, and company cultural fit directly influence the final grade.

4.2 Statistically Calibrated Selection Probability (Logistic Sigmoid)

Rather than displaying a misleading linear score as an offer probability, HireFit AI uses a calibrated logistic sigmoid function:

```
P_selection = 1 / (1 + exp(-k * (S_effective - S_0)))
```

Parameters:

- `S_effective = 0.85 * S_overall + 0.15 * S_resume_fit` (blends live interview performance with resume baseline)
- `S_0 = 70.0` (the empirical baseline threshold expected for competitive candidates at Tier-1 companies)
- `k = 0.08` (the logistic curve steepness calibration constant)

Mathematical Curve Mapping:

- `S_overall = 90.0` → `P_selection ≈ 83.2%` (High Confidence Candidate)
- `S_overall = 70.0` → `P_selection = 50.0%` (Competitive Midpoint / Borderline Candidate)
- `S_overall = 50.0` → `P_selection ≈ 16.8%` (Below Threshold / Needs Upskilling)
- `S_overall = 30.0` → `P_selection ≈ 3.9%` (Low Likelihood of Selection)

5. Master Viva & Technical Defense Q&A

Q1: How do you prevent LLM hallucinations during grading?

We enforce strict Pydantic v2 schemas (`AnswerEvaluationResult`) that reject conversational drift. In addition, all technical MCQs have pre-verified reference answer keys and conceptual rationales, meaning the LLM grades against objective rubrics rather than open-ended guessing.

Q2: Why did you choose FastAPI over Flask or Django?

FastAPI natively supports Python's asyncio event loop, allowing our `/sessions/{id}/submit-all` endpoint to execute 20 concurrent evaluation tasks in parallel via `asyncio.gather` . Flask is synchronous by default, and Django introduces heavy relational ORM overhead that doesn't fit our dynamic JSON document structures.

Q3: How does your web scraper handle bot blockers or Cloudflare?

We implement a graceful degradation architecture: the scraper attempts compliant httpx fetching with a 4-second timeout. If blocked (HTTP 403/429 or CAPTCHA), it seamlessly falls back to our synthetic LLM knowledge base profile, flagging the session with `is_limited_data=True` so the application never crashes.

Q4: Why use MongoDB instead of PostgreSQL or MySQL?

Interview sessions contain deeply nested, variable-length structures (20 turns, dynamic MCQ options, parsed resume entity dictionaries, ATS feedback). Storing this in relational SQL requires 6+ table joins. In MongoDB, an entire session is a single cohesive BSON document retrieved in sub-5ms with Motor async driver.

Q5: What is the significance of the Logistic Sigmoid formula?

A linear score (e.g. 70%) fails to capture hiring realities. Moving from 50% to 60% does not have the same impact as moving from 70% to 80% (crossing the hiring threshold). The sigmoid function centered at S_0=70 with k=0.08 models this exact non-linear competitive inflection point.

Q6: How does the system handle different seniority levels (Junior vs Senior vs Staff)?

The Role Analyzer parses seniority cues from the role title and JD, setting the initial difficulty parameter (Junior: Level 2, Mid: Level 3, Senior/Staff: Level 4). Senior tracks focus on distributed system trade-offs, circuit breakers, and SLA math rather than basic syntax.

Q7: How is candidate privacy and PII protected?

Candidate resumes are processed in memory bytes without saving raw files to public directories. The security module scrubs phone numbers, emails, and physical addresses before sending prompt payloads to external LLM providers.

Q8: What happens if MongoDB is offline on the evaluator's laptop?

We engineered an in-memory session store (`in_memory_sessions`) fallback in `backend/app/core/db.py` . If MongoDB is unavailable, all session creation, retrieval, and evaluations run seamlessly in Python memory without throwing connection exceptions.

Q9: What happens if the external LLM API key runs out of credits or has no internet?

Our provider-agnostic `LLMClient` wrapper includes an offline `MockLLMClient` fallback. It returns pre-compiled, mathematically consistent evaluations and ATS suggestions, guaranteeing a flawless presentation even with zero internet.

Q10: How does the ATS Resume Optimizer work?

It calculates the set intersection and semantic overlap between candidate resume skills and the target job description's core competencies. Missing keywords are flagged as high-priority gaps, and the LLM generates 3 STAR-formatted bullet points embedding those keywords into realistic achievements.